ST121 Assignment 1

[redacted]

17-02-2026

1. Vectors

i)

```
seq(from=-8, to = 13, by=3)
```

```
## [1] -8 -5 -2  1  4  7 10 13
```

ii)

```
rep(c(2,4,6,8,10,12), each = 3)
```

```
##  [1]  2  2  2  4  4  4  6  6  6  8  8  8 10 10 10 12 12 12
```

iii)

```
rep(c(9,6,3,0,-3,6), times = 6:1)
```

```
##  [1]  9  9  9  9  9  9  6  6  6  6  6  3  3  3  3  0  0  0 -3 -3  6
```

iv)

```
c((1:10)^seq(from = 2, to = 11, by = 1))
```

```
##  [1]           1           8          81        1024       15625
##  [6]      279936     5764801   134217728  3486784401 100000000000
```

v)

```
as.vector(rbind(-(3^(1:5)), 0, 3^(1:5)))
```

```
##  [1]   -3    0    3   -9    0    9  -27    0   27  -81    0   81 -243    0  243
```

vi)

```
as.vector(matrix(1:16, nrow = 4, byrow = T))
```

```
##  [1]  1  5  9 13  2  6 10 14  3  7 11 15  4  8 12 16
```

2. Random Numbers and Vectors

1)The random vector x is:

```
x<-rexp(1000,1/2)
```

2)i) The 200th, 400th, 600th, 800th, and 1000th elements are:

```
y <- c(x[200],x[400],x[600],x[800],x[1000])
y
```

```
## [1] 0.3324507 0.1854755 2.0039273 7.3720965 1.4058556
```

2)ii) The elements that are not from the first one hundred or last one hundred terms are:

```
z <-c(x[101:899])
```

2)iii) The number of elements between 6 and 9 is:

```
a<- length(x[x>6 & x<9])
a
```

```
## [1] 44
```

As a proportion, this is:

```
a_prop<-a/length(x)
a_prop
```

```
## [1] 0.044
```

2)iv) The theoretical answer to (iii) is:

```
a_theoretical <- pexp(9,rate = 1/2)-pexp(6,rate=1/2)
a_theoretical
```

```
## [1] 0.03867807
```

This is within a reasonable margin of the earlier result.

2)v) To generate a random sample with replacement from the elements of x that are less than 5, with size 500, it is written:

```
x_less_than_5 <- x[x<5]
b<- sample(x_less_than_5, size = 500,replace = T )
```

Note: The output is not printed due to length.

3. Short Mathematical Questions

1. Prime Numbers

a)This function returns TRUE if its argument is prime.

```
primefind<-function(n){
  if(length(n) !=1 || n%%1!=0){
    stop("Input must be integer")
  }
  if (n<=1){
    return(F)}
  else if (n==2){
    return(T)
  }
  else if (n==3){
    return(T)
  }
  for(i in 2:floor(sqrt(n))){
    if (n%%i ==0){
      return(F)
    }
  }
  return(T)
}
```

The function first ensures the input is an integer. Then, it deals with the cases of 1 (not prime), 2 (prime), and 3 (prime). After that, it tests the input's divisibility by doing modular division up to the highest factor below the floor of the square root of the input. This ensures the number is prime by testing all its possible divisors. Finally, the function will return "TRUE" if the input is prime and "FALSE" if the input is not prime.

b)This function accepts a list of integers and returns the prime numbers:

```
onlyprimes<-function(v){
  primes <-c()
  for(j in 1:length(v)){
    if(primefind(v[j])){
      primes <-c(primes, v[j])
    }
  }
  return(primes)
}
```

The function initializes a vector of the primes first before applying the earlier primefind function to each entry of the input. If the output of primefind is "TRUE", the entry gets appended to the primes vector. This vector is returned at the end of the function.

c)The prime numbers between 1 and 300 are:

```
onlyprimes(1:300)
```

```
##  [1]   2   3   5   7  11  13  17  19  23  29  31  37  41  43  47  53  59  61  67
## [20]  71  73  79  83  89  97 101 103 107 109 113 127 131 137 139 149 151 157 163
## [39] 167 173 179 181 191 193 197 199 211 223 227 229 233 239 241 251 257 263 269
## [58] 271 277 281 283 293
```

2. Square Numbers

i)The following function checks whether a positive integer is a square number:

```
squarenum<-function(w){
  w_sqrt<-sqrt(w)
  if(w<0||w%%1!=0){
    stop("Input must be a positive integer")
  }
  if (w_sqrt%%1!=0){
    return(F)}
  else{return(T)}
}
```

The above function checks the input is a positive integer. Then, it calculates the square root and checks if it is an integer. squarenum will return "TRUE" if the input is a square number, and "FALSE" otherwise.

ii) The following function takes a vector of positive integers and returns the square number entries:

```
onlysquares<-function(v){
  squares<-c()
  for(j in 1:length(v)){
    if (squarenum(v[j])){
      squares<-c(squares,v[j])
    }
  }
  return(squares)
}
```

The function initializes a vector for the output before applying the earlier squarenum function to each entry. If the output of the squarenum function is "TRUE," the entry gets added to the output vector. A vector of only the entries that are square numbers is returned.

iii) There are 69 square numbers between 1000 and 10000 inclusive, as found via:

```
length(onlysquares(1000:10000))
```

```
## [1] 69
```

iv) The theoretical result from the formula is:

$$\sum_{k=32}^{100} k^2 = \frac{(100)(101)(201)}{6} - \frac{(31)(32)(63)}{6} = 327934$$

The coded result is:

```
sum(onlysquares(1000:10000))
```

```
## [1] 327934
```

These results are the same, so the result is verified as required.

4. Longer Mathematical Questions

4.1 Preliminary Questions

a)The following function calculates the intercept and slope of a line P_1P_2 as well as the length of the segment from P_1 to P_2.

```
line.equation<-function(x1,y1,x2,y2){
  if (x1==x2||y1==y2){
    stop("Points must be distinct")
  }
  else{
  slope = (y2-y1)/(x2-x1)
  intercept = y2-slope*x2
  length= sqrt((x2-x1)^2+(y2-y1)^2)
  return = c(slope,intercept, length)}
  return(return)

}
```

b)The following function returns the intercept of two lines $y_1 = a_1x + b_1$ and $y_2 = a_2x + b_2$ (assuming they are not parallel).

```
intersection<-function(a1,b1,a2,b2){
  if(abs(a1-a2)<0.0001){
    stop("Lines cannot be parallel")
  }
  else{
  intersect.x = (b1-b2)/(a2-a1)
  intersect.y= a1*intersect.x+b1
  return = c(intersect.x, intersect.y)}
  return(return)
}
```

The function ensures the lines are not parallel (within some margin of error to account for computer accuracy). Then, it calculates the x and y intersect coordinates separately. Finally, it returns a vector containing the coordinates together.

c)The following function calculates the area of a triangle given its side lengths:

```
triangle.area<- function(a,b,c){
  s = (a+b+c)/2
  A2 = s*(s-a)*(s-b)*(s-c)
  A = sqrt(A2)
  return(A)
}
```

4.2 Triangles in the Unit Square

a)We set the following seed for consistency:

```
set.seed(3879)
```

Note: We used an additional digit to prevent the points being clustered in the corner

b)We begin by defining the points that are randomly generated for the dataframe. We include places for point I for use later.

```
x_a <-runif(1)
y_b<-runif(1)
x_c<-runif(1)
```

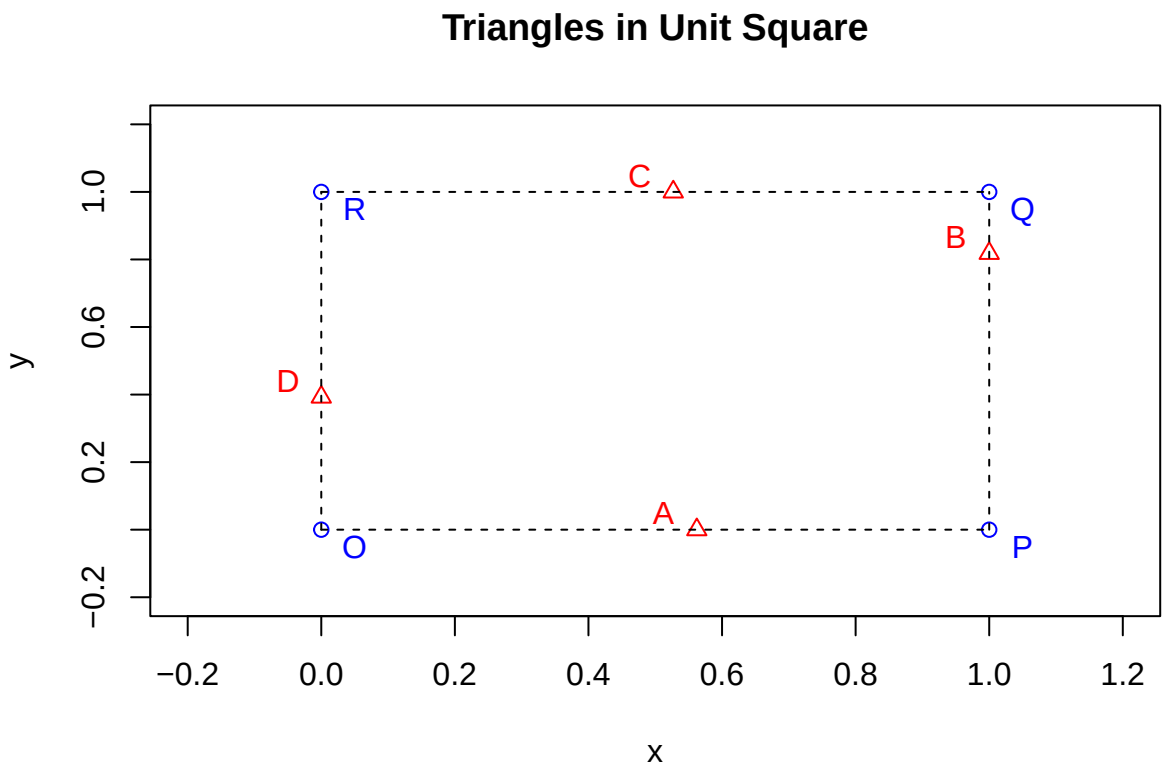
```
y_d<-runif(1)
x_i <- NA
y_i <-NA
```

Then we create the data frame.

```
Triangles_Data <-data.frame(
  Name = c("O", "P", "Q", "R", "A", "B", "C", "D", "I"),
  x_coordinates = c(0,1,1,0,x_a,1,x_c,0,x_i),
  y_coordinates = c(0,0,1,1,0,y_b,1,y_d, y_i)
)
```

c)Now we must create the plot. This plot includes: the points, the labels, and the outline of the square.

```
plot(Triangles_Data[1:4,2], Triangles_Data[1:4,3], pch = 1, col = "blue",
     main = "Triangles in Unit Square",
     xlab = "x", ylab = "y",
     xlim = c(-0.2, 1.2), ylim = c(-0.2, 1.2))
points(Triangles_Data[5:9, 2], Triangles_Data[5:9,3], pch=2, col = "red")
lines(seq(0,1,0.01), rep(0, times = length(seq(0,1,0.01))), type="l", lty = 2)
lines(seq(0,1,0.01), rep(1, times = length(seq(0,1,0.01))), type="l", lty = 2)
lines(rep(0, times = length(seq(0,1,0.01))), seq(0,1,0.01), type="l", lty = 2)
lines(rep(1, times = length(seq(0,1,0.01))), seq(0,1,0.01), type="l", lty = 2)
for(j in 1:4){
  text(Triangles_Data$x_coordinates[j]+0.05, Triangles_Data$y_coordinates[j]-0.05,
       labels = Triangles_Data$Name[j], col = "blue")
  }
for (j in 5:9){
  text(Triangles_Data$x_coordinates[j]-0.05, Triangles_Data$y_coordinates[j]+0.05,
       labels = Triangles_Data$Name[j], col = "red")
}
```



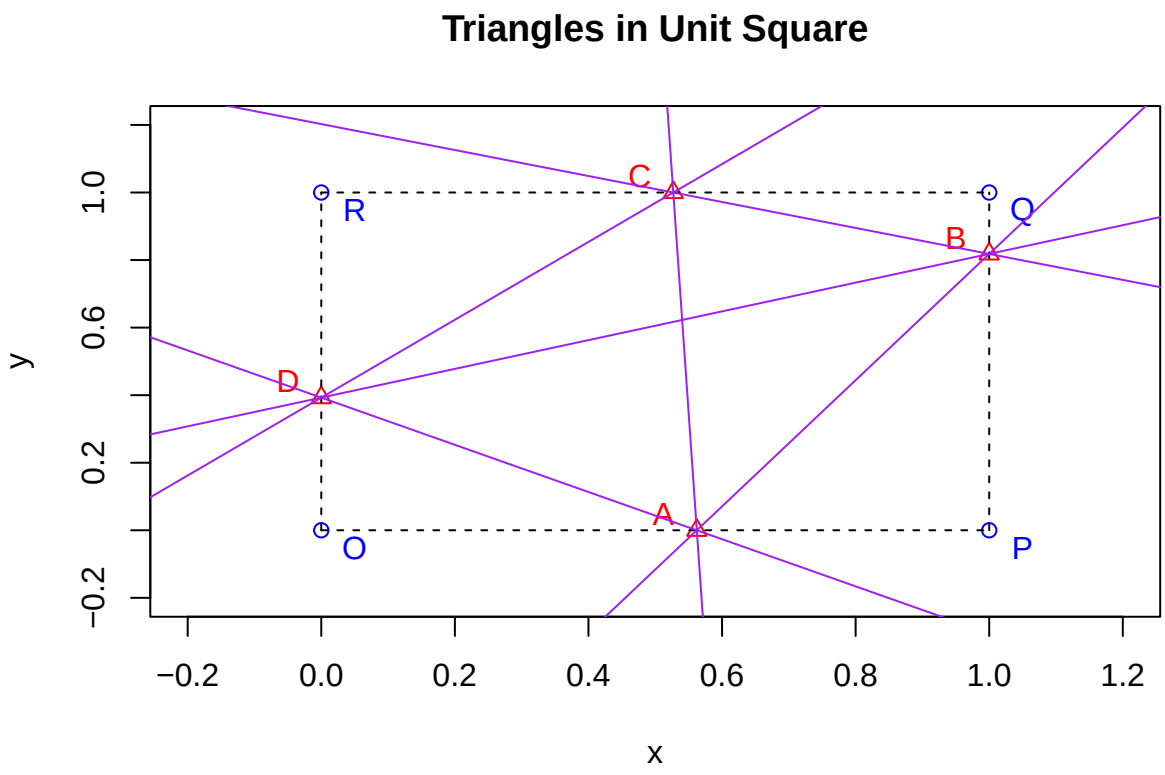
d)Now we add the lines connecting the vertices of ABCD, plotted in purple.

```
plot(Triangles_Data[1:4,2], Triangles_Data[1:4,3], pch = 1, col = "blue",
     main = "Triangles in Unit Square",
     xlab = "x", ylab = "y",
     xlim = c(-0.2, 1.2), ylim = c(-0.2, 1.2))
points(Triangles_Data[5:9, 2], Triangles_Data[5:9,3], pch=2, col = "red")
lines(seq(0,1,0.01), rep(0, times = length(seq(0,1,0.01))), type="l", lty = 2)
lines(seq(0,1,0.01), rep(1, times = length(seq(0,1,0.01))), type="l", lty = 2)
lines(rep(0, times = length(seq(0,1,0.01))), seq(0,1,0.01), type="l", lty = 2)
lines(rep(1, times = length(seq(0,1,0.01))), seq(0,1,0.01), type="l", lty = 2)
for(j in 1:4){
  text(Triangles_Data$x_coordinates[j]+0.05,
       Triangles_Data$y_coordinates[j]-0.05,
       labels = Triangles_Data$Name[j], col = "blue")
  }
for (j in 5:9){
  text(Triangles_Data$x_coordinates[j]-0.05,
       Triangles_Data$y_coordinates[j]+0.05,
       labels = Triangles_Data$Name[j], col = "red")
}

#NEW CODE

abline(y_b-((y_b-0)/(1-x_a))*1, (y_b-0)/(1-x_a), col = "purple") #AB
abline(1-((1-y_b)/(x_c-1))*x_c,(1-y_b)/(x_c-1), col = "purple") #BC
abline(y_d,(y_d-1)/(-x_c), col = "purple") #CD
```

```
abline(y_d,(-y_d)/(x_a), col = "purple") #DA
abline(1-((1)/(x_c-x_a))*x_c,(1)/(x_c-x_a), col = "purple") #AC
abline(y_d,(y_d-y_b)/-1, col = "purple") #DB
```



e)Now we must find I in order to plot and label it. First, define the lines AC and BD.

Line AC:

```
slope_ac = ((Triangles_Data$y_coordinates[7]-Triangles_Data$y_coordinates[5])/
              (Triangles_Data$x_coordinates[7]-Triangles_Data$x_coordinates[5]))
intercept_ac = (Triangles_Data$y_coordinates[5]-
                  slope_ac*Triangles_Data$x_coordinates[5])
```

Line BD:

```
slope_bd = ((Triangles_Data$y_coordinates[8]-Triangles_Data$y_coordinates[6])/
              (Triangles_Data$x_coordinates[8]-Triangles_Data$x_coordinates[6]))
intercept_bd = (Triangles_Data$y_coordinates[6]-
                  slope_bd*Triangles_Data$x_coordinates[6])
```

Now we find the intersection point:

```
x_i<- (intercept_ac-intercept_bd)/(slope_bd-slope_ac)
y_i <- slope_bd*x_i+intercept_bd
```

Update the data frame with these new coordinates for I.

```
Triangles_Data$x_coordinates[9]<-x_i
Triangles_Data$y_coordinates[9]<-y_i
```

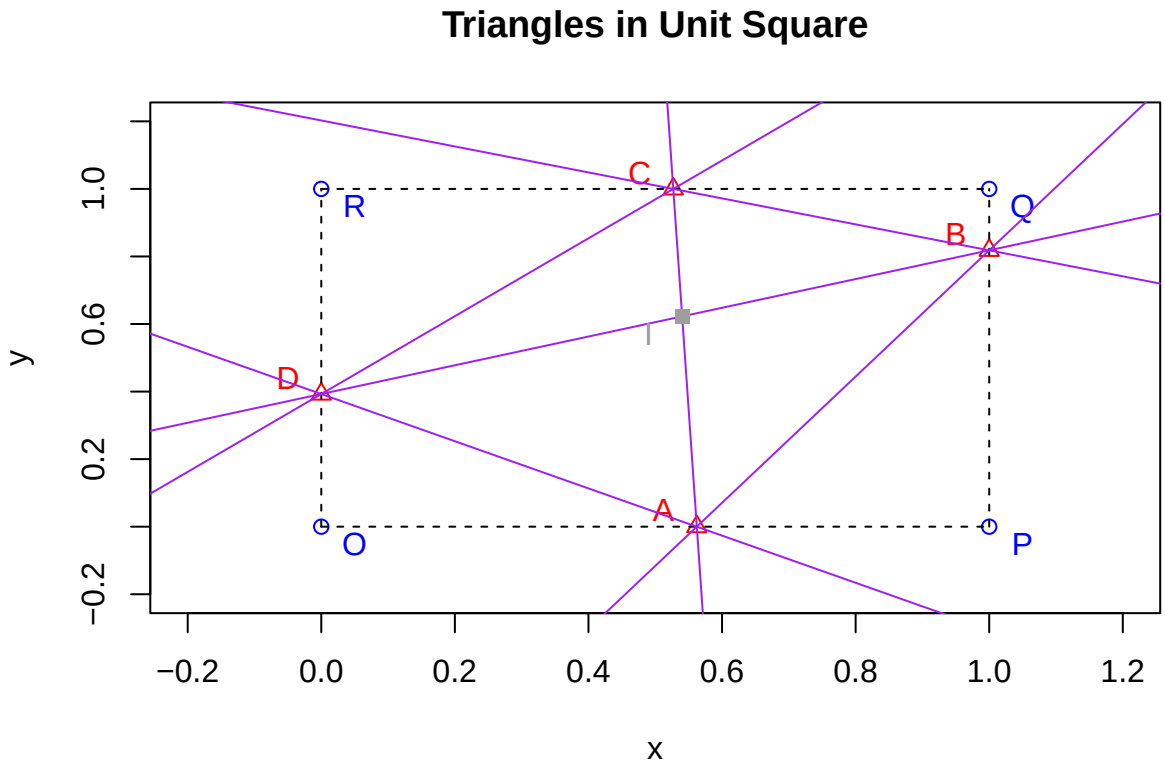
We now plot and label the point I on the graph.

```
plot(Triangles_Data[1:4,2], Triangles_Data[1:4,3], pch = 1, col = "blue",
     main = "Triangles in Unit Square",
     xlab = "x", ylab = "y",
     xlim = c(-0.2, 1.2), ylim = c(-0.2, 1.2))
points(Triangles_Data[5:8, 2], Triangles_Data[5:8,3], pch=2, col = "red")
lines(seq(0,1,0.01), rep(0, times = length(seq(0,1,0.01))), type="l", lty = 2)
lines(seq(0,1,0.01), rep(1, times = length(seq(0,1,0.01))), type="l", lty = 2)
lines(rep(0, times = length(seq(0,1,0.01))), seq(0,1,0.01), type="l", lty = 2)
lines(rep(1, times = length(seq(0,1,0.01))), seq(0,1,0.01), type="l", lty = 2)
for(j in 1:4){
  text(Triangles_Data$x_coordinates[j]+0.05,
       Triangles_Data$y_coordinates[j]-0.05,
       labels = Triangles_Data$Name[j], col = "blue")
  }
for (j in 5:8){
  text(Triangles_Data$x_coordinates[j]-0.05,
       Triangles_Data$y_coordinates[j]+0.05,
       labels = Triangles_Data$Name[j], col = "red")
}


abline(y_b-((y_b-0)/(1-x_a))*1, (y_b-0)/(1-x_a), col = "purple") #AB
abline(1-((1-y_b)/(x_c-1))*x_c,(1-y_b)/(x_c-1), col = "purple") #BC
abline(y_d,(y_d-1)/(-x_c), col = "purple") #CD
abline(y_d,(-y_d)/(x_a), col = "purple") #DA
abline(1-((1)/(x_c-x_a))*x_c,(1)/(x_c-x_a), col = "purple") #AC
abline(y_d,(y_d-y_b)/-1, col = "purple") #DB


#NEW CODE

#label on graph
points(Triangles_Data$x_coordinates[9], Triangles_Data$y_coordinates[9],
       col = "006600", pch = 15)
text(Triangles_Data$x_coordinates[9]-0.05, Triangles_Data$y_coordinates[9]-0.05,
     label = Triangles_Data$Name[9], col = "006600")
```



For visual reference, we can also add the lines AC and BD to ensure the placement of I is reasonable:

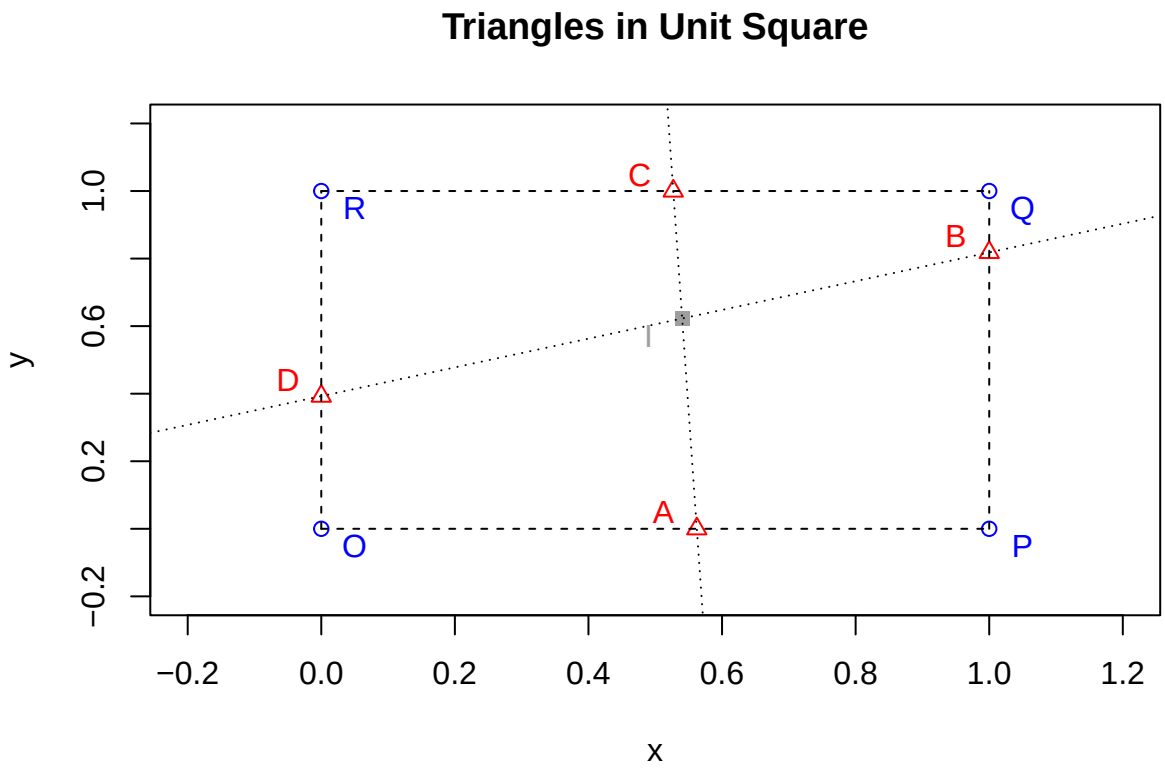
```
plot(Triangles_Data[1:4,2], Triangles_Data[1:4,3], pch = 1, col = "blue",
     main = "Triangles in Unit Square",
     xlab = "x", ylab = "y",
     xlim = c(-0.2, 1.2), ylim = c(-0.2, 1.2))
points(Triangles_Data[5:8, 2], Triangles_Data[5:8,3], pch=2, col = "red")
lines(seq(0,1,0.01), rep(0, times = length(seq(0,1,0.01))), type="l", lty = 2)
lines(seq(0,1,0.01), rep(1, times = length(seq(0,1,0.01))), type="l", lty = 2)
lines(rep(0, times = length(seq(0,1,0.01))), seq(0,1,0.01), type="l", lty = 2)
lines(rep(1, times = length(seq(0,1,0.01))), seq(0,1,0.01), type="l", lty = 2)
for(j in 1:4){
  text(Triangles_Data$x_coordinates[j]+0.05,
       Triangles_Data$y_coordinates[j]-0.05,
       labels = Triangles_Data$Name[j], col = "blue")
  }
for (j in 5:8){
  text(Triangles_Data$x_coordinates[j]-0.05,
       Triangles_Data$y_coordinates[j]+0.05,
       labels = Triangles_Data$Name[j], col = "red")

}

points(Triangles_Data$x_coordinates[9],
       Triangles_Data$y_coordinates[9],
       col = "006600", pch = 15)
text(Triangles_Data$x_coordinates[9]-0.05,
```

```
     Triangles_Data$y_coordinates[9]-0.05,
     label = Triangles_Data$Name[9], col = "006600")

#NEW CODE
#check intercept of these lines
abline(intercept_ac, slope_ac, lty = "dotted")
abline(intercept_bd, slope_bd, lty = 'dotted')
```



It seems to be reasonably placed, so the code above can be considered accurate.

f) We must construct a function that computes the area of a triangle, given its name.

```
triangle.area.by.name<-function(triangle.name){
  p1<-substr(triangle.name,1,1)
  p2<-substr(triangle.name, 2,2)
  p3<-substr(triangle.name,3,3)

  q1<-Triangles_Data[Triangles_Data$Name ==p1, c("x_coordinates","y_coordinates")]
  q2<-Triangles_Data[Triangles_Data$Name ==p2, c("x_coordinates","y_coordinates")]
  q3<-Triangles_Data[Triangles_Data$Name ==p3, c("x_coordinates","y_coordinates")]

  x1<-q1$x_coordinates; y1<-q1$y_coordinates
  x2<-q2$x_coordinates; y2<-q2$y_coordinates
  x3<-q3$x_coordinates; y3<-q3$y_coordinates

  area<-0.5*abs(x1*(y2-y3)+x2*(y3-y1)+x3*(y1-y2))
```

```
  return(area)
}
```

The function above works as follows:
1) It extracts the first, second, and third point names from the input string.
2) It determines the coordinate pair of the corresponding point in the dataframe.
3) Those points are saved as individual x and y values.
4) The area is calculated using the determinant method.

To sanity-check our function, we use the following triangle, OPR, which is given to have area of 0.5.

```
triangle.area.by.name("OPR")
```

```
## [1] 0.5
```

The function agrees with our calculation, so it can be considered trustworthy.

g)First, we must determine the eight elementary triangles and save them as a vector.

```
elementary_triangles <- c("OAI", "ADI", "ABI", "BPI", "BCI", "CQI", "CDI", "DRI")
```

Next, we initialize a vector that will contain the areas of each triangle. Then the code will loop through each triangle and determine its area using the function in (f).

```
areas_elementary <-c()
for(i in 1:8){
  new_area<-triangle.area.by.name(elementary_triangles[i])
  areas_elementary<-c(areas_elementary, new_area)
}
```

Now, we must find the smallest area and its corresponding triangle.

The smallest triangle is:

```
elementary_triangles[which.min(areas_elementary)]
```

```
## [1] "BCI"
```

with area:

```
areas_elementary[which.min(areas_elementary)]
```

```
## [1] 0.08805397
```

By manual inspection, this is shown to be correct, increasing trustworthiness of the code.